Chapter 04

Navigation and Multi-Page Applications

2 lessons • 25 hr

elevify

Chapter 04

2 lessons • 25 hr

Chapter 04 • Navigation and Multi-Page Applications

Lesson 01 • Navigation Rail and Bottom Navigation Bar

14 pages • 23 min

elevify

Lesson 01 • Navigation Rail and Bottom Navigation Bar

23 min • 14 pages

◎ What you will learn

Implements NavigationRail and NavigationBar for persistent navigation patterns. These components are standard in professional desktop and mobile apps.

Navigation Rail and Bottom Navigation Bar

Summary

Article Truncation

The generated article is too short and incomplete, cutting off mid-sentence. This premature ending means the content is not fully delivered. The reader is left without a complete understanding of the topics that were meant to be covered. The article fails to provide the intended information and is missing many critical sections that are essential for understanding the material. The truncation is a major flaw that renders the article unusable for learning or reference.

Missing Code and NavigationRail

The article omits detailed code examples, which are crucial for practical learning and implementation. Also missing are the NavigationRail's three key pieces and the on_change handler. These are fundamental components for implementing navigation in the code. Without them, the reader cannot follow the implementation steps. The absence of these sections severely limits the article's usefulness and leaves the reader without necessary guidance.

Layout and NavigationBar Omitted

The article lacks layout instructions and NavigationBar implementation details. Additionally, the sync problem and its solution are not covered. These are important for understanding how to structure the app and handle synchronization issues. The reader misses out on key troubleshooting information that is essential for a complete development process. This omission hinders the ability to build a fully functional application.

Routing and Adaptive Navigation Missing

The article does not include routing integration and adaptive navigation. These topics are essential for building responsive and navigable applications that adapt to different screen sizes. Also missing are common pitfalls that developers face. Without these, the reader may encounter errors or unoptimized behavior. The article thus fails to provide comprehensive guidance on modern navigation patterns.

Recap and Key Concepts Absent

The article omits the recap section and key topics and concepts from the transcription. The recap would have summarized the main points, and the key concepts are foundational for understanding the material. Without them, the reader cannot review or retain the important information. The article is incomplete and unsatisfactory, lacking the closure and reinforcement needed for effective learning.

NOTES

Navigation Rail and Bottom Navigation Bar

Transcript

{ "message" : "The generated text is too short: The generated article is incomplete, cutting off mid-sentence. It omits critical sections including detailed code examples, the NavigationRail's three key pieces, the on_change handler, layout instructions, NavigationBar implementation, the sync problem and solution, routing integration, adaptive navigation, common pitfalls, and the recap. Key topics and concepts from the transcription are missing.", "status" : "ERR_RETRYABLE" }

NOTES

Further study materials Lesson 1

Content type: Text

Customizing Flutter NavigationRail: Properties, Themes, and Layout Workarounds

Available at https://medium.com/flutter-community/flutter-navigationrail-widget-new-in-v1-17-4b8c5416a081

Full Content

NOTES

Summary

Introduction to NavigationRail

Flutter v1.17 introduced the NavigationRail widget, an attractive alternative to BottomNavigationBar. Developers often stick with familiar tools, but Flutter's packaging encourages exploring new widgets. NavigationRail can be used alone or with BottomNavigationBar for seamless device experiences. It is a material widget displayed at the left or right of an app, typically for navigating between three to five pages or fragments.

What is NavigationRail?

NavigationRail is a creative alternative to BottomNavigationBar, introduced in Flutter's first stable release of 2020. It is a material widget meant to be placed on the left or right side of an app to navigate between a small number of pages, typically three to five. The Material Design team has laid out explicit principles, and designers on Dribbble have created stunning examples. The widget can be used in tandem with BottomNavigationBar for a seamless cross-device experience.

Typical Structure of NavigationRail

Unlike AppBar or BottomNavigationBar, NavigationRail does not have a specific slot in Scaffold. It is usually placed as the first or last element of a Row that defines the Scaffold body. The rail takes the extreme left or right slot, separated from main content by a VerticalDivider or by using the elevation property. The main content should be wrapped in Expanded to fill remaining space. The code example shows how to set up NavigationRail with destinations and a selectedIndex state.

NavigationRail Properties Explained

NavigationRail properties include destinations (list of NavigationRailDestination objects), selectedIconTheme and unselectedIconTheme for icon customization, selectedLabelTextStyle and unselectedLabelTextStyle for label text styles, labelType (none, selected, all) to control label visibility, minWidth (default 72) for compact mode, groupAlignment (range -1 to 1) for vertical alignment, leading and trailing widgets, selectedIndex, and onDestinationSelected callback.

Further study materials Lesson 1

Summary

NavigationRailDestination has icon, selectedIcon (optional), and label (mandatory).

Code Example and Missing Features

The complete code for a sample NavigationRail is available online. One missing feature is inter-destination padding, which can be added hackily by wrapping icon in Padding. Another is vertical text customization using RotatedBox on the label. The author finds the widget full-featured but wishes for a global vertical padding property. Overall, NavigationRail is a wonderful addition to Flutter's widget library, enabling creative navigation patterns.

NOTES

Further study materials Lesson 1

Transcript

Flutter: NavigationRail Widget – New in v1.17

*[Image: https://miro.medium.com/1*0gq-MSBskcUrNceKWH3bng.png]*

As a developer who has active apps on app stores and hundreds of thousands of users who depend on them, it's particularly difficult to keep track of every new framework or API being launched every day. We tend to be happy with the tools we have mastered already and only look to change them when absolutely necessary.

But the experience has been a tad different with Flutter. The way Team Flutter has packaged and presented these tools (widgets in this case), I have often found myself looking for ways to include these attractive new tools in my workflow. One such attractive tool was added to the inventory with the v1.17 stable release of Flutter a couple of days ago. No prizes for guessing, I am talking about the `NavigationRail` widget.

Anyway, enough talk about artisanry. ? Time for us to get down to business.

What is NavigationRail?

Flutter's first stable release of 2020 brought plenty to talk about, ranging from performance improvements to massive improvements in tooling. But what caught my eye was this li'l widget called `NavigationRail`, which can be termed as a creative alternative to `BottomNavigationBar`. The two can also be used in tandem to create a seamless experience across devices.

As described here, it's a material widget that is meant to be displayed at the left or right of an app to navigate between a small number of pages or fragments (if I may call them that), typically between three and five.

The principles and rules are laid out quite explicitly by the Material Design team, but ever since its inception, the designer community, especially on Dribbble, seems to have taken a special liking for the component and has created some stunning designs using it. Have a look.

*[Image: https://miro.medium.com/1*mVWwGnY30d-Lh-2irU7T7g.jpeg]*

*[Image: https://miro.medium.com/1*AcBCJEf_T4qFuSm87a4kGQ.jpeg]*

[Image: Credits: left to right: @purrweb @tugboi @phamhuyds]

Typical Structure

Unlike the `AppBar` or the `BottomNavigationBar`, the `NavigationRail` doesn't have a specific slot allotted to itself in a `Scaffold`. A navigation rail is usually used as the first or last

NOTES

Further study materials Lesson 1

Transcript

element of a Row which defines the app's Scaffold *body*.

In the Row assigned to the `Scaffold` *body*, the rail takes up the extreme left or right slot. The main content and the rail are generally separated by a `VerticalDivider`. However, the **elevation** property of the `NavigationRail` can also be used for the same. The main content needs to be wrapped in an `Expanded` such that it takes up all of the remaining space.

*[Image: https://miro.medium.com/1*3bvnYExEQigutvlsYRNFaQ.png]*

```
Scaffold(
body: Row(
children: <Widget>[
NavigationRail(
selectedIndex: _selectedIndex,
onDestinationSelected: (int index) {
setState(() {
_selectedIndex = index;
});
},
labelType: NavigationRailLabelType.selected,
destinations: [
NavigationRailDestination(
icon: Icon(Icons.favorite_border),
selectedIcon: Icon(Icons.favorite),
label: Text('First'),
),
NavigationRailDestination(
icon: Icon(Icons.bookmark_border),
selectedIcon: Icon(Icons.book),
label: Text('Second'),
),
NavigationRailDestination(
icon: Icon(Icons.star_border),
selectedIcon: Icon(Icons.star),
label: Text('Third'),
),
],
),
VerticalDivider(thickness: 1, width: 1),
// This is the main content.
Expanded(
child: Center(
child: Text('selectedIndex: $_selectedIndex'),
),
)
],
),
);
```

Widget Properties: Explained

Okay, so the designs look cool and we can't wait to create something like this ourselves. So how do we do it? We'll try and understand the properties one by one. But before that, there's one

NOTES

Further study materials Lesson 1

Transcript

class that needs introduction. It's the `NavigationRailDestination` class.

What is *NavigationRailDestination*?

[Image: Credits: material.io]

It is a model class used to create tappable destinations in the NavigationRail. It holds the data to represent one destination view or fragment. Mind you, it is not a widget class. It contains three properties:

1. **icon**: supposed to take an Icon widget (generally the outlined state) and must always be non-null.
2. **selectedIcon**: It is an optional field that can be used to provide an alternative (generally filled) state of the icon. This can be null.
3. **label**: A mandatory field that is to be assigned a Text widget. It must always be non-null. The label can be shown or hidden using the **labelType** property of the `NavigationRail`.

The usual `NavigationRailDestination` object looks like this.

```
NavigationRailDestination(
icon: Icon(Icons.favorite_border),
selectedIcon: Icon(Icons.favorite),
label: Text('First'),
),
```

You'll soon realize why this class was explained at this point.

Let's start with the NavigationRail properties

- **destinations**: This property takes a list of `NavigationRailDestination` objects. The list should contain two or more items. Standard stuff.
- **selectedIconTheme** and **unselectedIconTheme**: Pretty self-explanatory. Using these two properties, you can define the **color**, **opacity**, and **size** of the icons of the `NavigationRailDestination` objects, when they are selected and when they are not. This property takes `IconThemeData` objects.
- **selectedLabelTextStyle** and **unselectedLabelTextStyle**: Again, using these two properties, you can customize the look and feel of the text labels for selected and unselected states. You need to assign `TextStyle` objects to these properties.
- **labelType**: takes an enum value of type `NavigationRailLabelType`, which gives you three choices: `NavigationRailLabelType.none`, `NavigationRailLabelType.selected`, `NavigationRailLabelType.all`. Check out the illustration below. It should explain how the property can be used to hide or show the text labels.

NOTES

Further study materials Lesson 1

Transcript

[Image: Illustration for labelType]

• **minWidth**: The smallest possible width for the rail regardless of the destination's icon or label size. The default value is 72. When set to 56 along with **labelType** set to *none*, it forms a compact rail.

[Image: Illustration for minWidth]

• **groupAlignment**: This property is used to set the vertical alignment of the rail destinations. It takes a value between -1.0 and 1.0. The default value is -1.0, which sets the alignment to the top. A value of 0.0 aligns it to the center. A value of 1.0 aligns the rail items to the bottom. Arbitrary values within that range can also be assigned.

[Image: Illustration for groupAlignment]

• **leading** and **trailing**: These properties work in the same way as they do in other widgets like ListTile, AppBar, etc. They take any widget objects. Hence, in the design below, it was easy to nest multiple widgets inside a `Column` and pass them as leading or trailing. However, I could not get the trailing widgets to align to the absolute bottom of the rail when **groupAlignment** was set to something other than bottom. They would only trail the rail destinations.

*[Image: https://miro.medium.com/1*DVJU89gUR_yF7S8L3eqWdQ.png]*

• **selectedIndex**: The index into destinations for the current selected `NavigationRailDestination`.

• **onDestinationSelected**: This callback is called when one of the destinations is selected. The stateful widget that creates the navigation rail needs to keep track of the index of the selected `NavigationRailDestination` and call `setState` to rebuild the navigation rail with the new `selectedIndex`.

That's all regarding the important properties you need to know.

The Code

*[Image: https://miro.medium.com/1*9ZSLgdeecrpD-1vZJPdxBw.gif]*

Not embedding the code here directly, as it takes up unnecessary space. You can find the complete code for this screen here.

Some Missing Features

Unlike a number of other widgets, the implementation of `NavigationRail` felt quite full-featured, with a handful of customization options. Yet one of the most important things I missed was a global property for **Inter-RailDestination padding.** I understand it not being there for a `BottomNavigationBar`, but for a widget that will receive tons of vertical space, it would have been nice to have. I was able to create a hacky workaround for it,

NOTES

Further study materials Lesson 1

Transcript

but it's not the most ideal solution. The padding obviously only works when *labelType* is set to *All.*

```
NavigationRailDestination(
icon: Padding(
padding: const EdgeInsets.symmetric(vertical: 24),
child: Icon(Icons.av_timer)),
label: SizedBox.shrink(),
),
```

Vertical Text Customization

With the following code, you can achieve vertical text direction, which can be used even in the compact mode of the NavigationRail.

```
NavigationRailDestination(
icon: SizedBox.shrink(),
label: Padding(
padding: const EdgeInsets.symmetric(vertical: 24),
child: RotatedBox(
quarterTurns: -1,
child: Text("text"),
),
),
```

Phew, today was supposed to be my day off from work. But here I am, having written more than 1300 words for this article, paired with the research, the code, the composition, and much more. I already had a couple of articles planned, but I took this up because I find this new design pattern incredibly intriguing, and I feel that the widget is a wonderful addition to an already amazing library. Moreover, it always feels nice to give a little something back to the community. 👍

If this article helps you, consider clapping ? +50 times; it will encourage me to keep investing my time in such community work.

Also, please consider tagging me on Twitter to show me the wonderful user experiences inspired by this article.

*" **Ankit Chowdhury***

Until next time, see ya. ?

*" **Flutter Community***

NOTES

Exercises • Lesson 1

EXERCISE 1 • True or false

CONTEXT

In Flutter, different navigation components are used for various platform-specific layouts.

QUESTION • Indicate whether the information is true or false

NavigationRail consists of three key pieces.

◯ TRUE ◯ FALSE

NOTES

Exercises • Lesson 1

EXERCISE 2 • Multiple choice

CONTEXT

The lesson discussed the components of a Flutter navigation system, highlighting a specific component that is built from three key pieces.

QUESTION • Select the correct option

Which component is described as consisting of three key pieces?

A NavigationBar

B AppBar

C Drawer

D NavigationRail

NOTES

Exercises • Lesson 1

EXERCISE 3 • True or false

CONTEXT

NavigationRail is a material widget introduced in Flutter v1.17, used for navigation within an app. It offers various properties to customize its appearance, including minWidth, labelType, and groupAlignment.

QUESTION • Indicate whether the information is true or false

The minWidth property of NavigationRail has a default value of 56.

◯ TRUE ◯ FALSE

NOTES

Exercises • Lesson 1

EXERCISE 4 • Multiple choice

CONTEXT

The NavigationRail widget has properties to customize its appearance and behavior, including how destination labels are displayed.

QUESTION • Select the correct option

Which value of the NavigationRailLabelType enum displays labels only for the currently selected destination?

A NavigationRailLabelType.all

B NavigationRailLabelType.none

C NavigationRailLabelType.one

D NavigationRailLabelType.selected

NOTES

Exercise Answers

LESSON 1

EX.01 • True or false

True.

NavigationRail is designed with three main components: leading, body, and trailing, which together form the navigation structure.

I got it right I got it wrong

EX.02 • Multiple choice

Option D — “NavigationRail”

The lesson content explicitly mentions that NavigationRail consists of three key pieces, making it the correct answer.

I got it right I got it wrong

EX.03 • True or false

False.

The default minWidth is 72, not 56. Setting minWidth to 56 with labelType none creates a compact rail, but the default is 72.

I got it right I got it wrong

EX.04 • Multiple choice

Option D — “NavigationRailLabelType.selected”

The lesson states that setting labelType to NavigationRailLabelType.selected shows labels only for the selected destination, while none hides all and all shows all labels.

I got it right I got it wrong

Chapter 04 • Navigation and Multi-Page Applications

Practical project of the chapter

15 pages

elevify

Practical project of the chapter

15 pages

Navigation and Multi-Page Applications

Driving Question

Build a functional Flet app that demonstrates multi-page navigation using NavigationRail, BottomNavigationBar, and URL-based routing, with placeholder views for Home, Learn, and Settings.

Production Format

Argumentative essay — 800 to 1500 words.

Context and Provocation

Objective

Produce a Flet application with three placeholder views (Home, Learn, Settings) and working navigation using NavigationRail for desktop and BottomNavigationBar for mobile. You will exercise view creation, the view stack, route-based navigation, and data passing between views. This scaffold will later host your journal entries, mood cards, and theme toggle.

Creative Brief

Your friend, who needs the personal journal app, is eager to see the basic “bones” before you pour in all the features. “I want to click around and feel how it moves—just a clean shell that switches between tabs on my laptop and adapts to my phone. Make it feel like a real app already!” This milestone is your chance to design a navigation experience that feels native and intuitive. Later, you’ll plug in the journal content, but for now, impress them with smooth transitions and a polished sidebar.

Objective

Produce a Flet application with three placeholder views (Home, Learn, Settings) and working navigation using NavigationRail for desktop and BottomNavigationBar for mobile. You will exercise view creation, the view stack, route-based navigation, and data passing between views. This scaffold will later host your journal entries, mood cards, and theme toggle.

Creative Brief

Your friend, who needs the personal journal app, is eager to see the basic “bones” before you pour in all the features. “I want to click around and feel how it moves—just a clean shell that switches between tabs on my laptop and adapts to my phone. Make it feel like a real app already!” This milestone is your chance to design a navigation experience that feels native and intuitive. Later, you’ll plug in the journal content, but for now, impress them with smooth transitions and a polished sidebar.

Materials and Resources

- Computer with Python 3.7 or later installed
- Flet package installed (pip install flet)
- Code editor (VS Code, PyCharm, or similar)
- Optional: A mobile device or emulator to test responsive layout
- Reference: Flet documentation on View, NavigationRail, NavigationBar, and routing (available in the course resources)

Execution Steps

1. Create a new Python file named main.py and import Flet.

- Professional insider tip: Use a virtual environment to keep dependencies isolated.
- Common mistake to avoid: Forgetting to call ft.app(target=main) at the bottom of the script.

2. Define a main(page: ft.Page) function and set initial properties (title, theme, etc.).

- Professional insider tip: Set page.theme_mode = ft.ThemeMode.SYSTEM to support light/dark out of the box.
- Common mistake to avoid: Not setting page.title early enough, which leads to a generic window title.

3. Create three placeholder view functions (home_view, learn_view, settings_view) that return ft.View objects with a simple Text control identifying the page.

- Professional insider tip: Use ft.SafeArea and padding for a consistent look across devices.
- Common mistake to avoid: Returning a plain Control instead of a ft.View — you need the AppBar integration.

4. Build a NavigationRail with three destinations (Home, Learn, Settings) and add it to the page via page.navigation_bar for desktop.

- Professional insider tip: Set label_behavior=ft.NavigationRailLabelBehavior.ALWAYS_SHOW for clarity.
- Common mistake to avoid: Forgetting to bind the on_change event handler to update the selected index and route.

5. Create a BottomNavigationBar with the same destinations and assign it to page.navigation_bar conditionally when page.width < 600 (mobile).

- Professional insider tip: Wrap the assignment in page.on_resize to make it responsive.
- Common mistake to avoid: Using the same control instance for both NavigationRail and BottomNavigationBar — they must be separate.

6. Implement the route_change handler to swap views based on page.route and call page.update().

- Professional insider tip: Use a dictionary mapping routes to view builders for clean code.
- Common mistake to avoid: Neglecting to pop the previous view; use page.views.pop() before appending

the new one unless you want a stack.

7. Add an AppBar to the Settings view with a back button that pushes a view onto the stack, and handle its on_click to pop.

• Professional insider tip: Set automatically_imply_leading=False on the AppBar to control the back button yourself.

• Common mistake to avoid: Popping views from an empty stack — check page.views length first.

8. Test your app on different window sizes to verify the navigation adapts and all routes work. Record a short video of the result.

• Professional insider tip: Use Flet's hot reload (flet run main.py) to speed up testing.

• Common mistake to avoid: Skipping mobile-width testing — a NavigationRail on a phone screen looks broken.

Self-Assessment Checklist

• The app launches without errors and displays three navigation destinations.

• On wide screens (>= 600px), a NavigationRail is visible and highlights the active view.

• On narrow screens (< 600px), a BottomNavigationBar replaces the rail.

• Clicking navigation items switches the view and updates the URL route.

• The Settings view shows an AppBar with a back button that returns to the previous view.

• Navigating to an unknown route (e.g., /missing) shows a 404 placeholder.

• Code is cleanly structured into separate functions for each view and the main setup.

Bonus Challenge

Add a fourth route "/mood" that displays a simple mood selector (slider or radio buttons) and passes the chosen mood back to the Home view via a route parameter. Show the passed mood as a welcome message on the Home screen.

Submission Format

Accepted formats:

• video (.mp4)

• text (.txt)

Minimum delivery:

• one video file (screen recording of the app demonstrating navigation on both desktop and mobile widths) and one text file containing the complete Python code.

Development Guide

1. Research Phase
Look for examples of small apartment designs on sites like ArchDaily or Dezeen. Read news articles on micro-apartments to see both praise and criticism. Note how different designs handle the space-saving vs. comfort tradeoff. Use the course chapters on apartment planning and 3D modeling as your main guides.
2. Organization Phase
Start by stating your main argument — your position on balancing efficiency and comfort. Then outline three to four key points that support your view, using the mandatory references. For each point, plan what evidence or examples you'll use. Also think about one counterargument and how you'll respond to it.
3. Writing Phase
Write an introduction that presents the problem and your thesis. In the body paragraphs, each should focus on one key point — explaining a course concept and applying it to your design example. Use simple language: imagine explaining to a friend. Include specific examples, like how you'd place a bed or choose lighting. End with a conclusion that summarizes your argument and suggests practical tips for designers.
4. Review Phase
Check that your essay clearly states your position. Verify that you've used all four mandatory references correctly. Read it aloud to see if the logic flows smoothly. Ask yourself: would a beginner understand this? Remove any jargon or complex terms without explanation.

Self-Assessment Rubric

1. App Launches & Shows Destinations

• Insufficient — App crashes on launch, or navigation bar/rail is missing/empty.

• Adequate — App starts; destinations are visible but may lack icons or proper spacing; no crashes.

• Excellent — App starts with no errors; three labelled destinations (with icons) are clearly visible and styled appropriately.

2. Responsive Navigation (Rail for Wide)

• Insufficient — Rail never appears, or appears on narrow screens, or both rail and bar show simultaneously.

• Adequate — Rail visible on wide screens but highlighting may lag or not update on resize without manual trigger.

• Excellent — NavigationRail appears precisely when width >= 600px, highlights the active view, and adapts smoothly on window resize.

3. Responsive Navigation (Bar for Narrow)

• Insufficient — Bar missing entirely, or constantly overlaps with the rail.

• Adequate — Bar appears on narrow screens initially but does not respond to resize events; may require restart.

• Excellent — BottomNavigationBar appears when width < 600px, mirrors the rail’s destinations, highlights active view, and transitions responsively on resize.

4. Navigation Links Work & Update Route

• Insufficient — Clicking does nothing, changes to wrong view, or route logic missing.

• Adequate — View changes on click, but route may not update visibly or only works from click, not direct URL.

• Excellent — Each navigation item click instantly switches the main view and updates the browser-style route (URL). Routes can also be navigated by typing manually.

5. Settings Back Button

• Insufficient — No AppBar, back button crashes, or cannot return to previous view.

• Adequate — Back button present but always goes to Home instead of previous; or AppBar appears in all views.

• Excellent — Settings view has a clean AppBar with a back button (arrow icon) that pops the view stack and returns to the exact previous view. No crashes.

6. 404 Placeholder for Unknown Routes

• Insufficient — Unknown route leaves app blank, shows last view, or crashes.

• Adequate — Shows plain "Not Found" text, but without a proper AppBar or styling.

• Excellent — Navigating to any undefined route (e.g., /missing) shows a friendly 404 view with text and does not crash.

7. Code Cleanliness & Structure

• Insufficient — All logic inside 'main', no view functions, or returns wrong control types; messy.

• Adequate — Views are separate functions, but main function is long or some duplication exists.

• Excellent — Code is modular: view functions separated, route handler distinct, clear variable names, concise comments. No code duplication.

8. Video Demonstration Quality

• Insufficient — No video submitted, or video does not display the app functioning.

• Adequate — Video shows basic navigation but does not demonstrate window resize explicitly or misses 404 test.

• Excellent — Video (.mp4) clearly shows the app running on both wide and narrow widths, navigation through all routes, back button, and 404 test. Narration or captions helpful.

Model Response & Assessment Reference

> *This guide presents possible paths, not single answers. An excellent essay may defend any of the theses below — or an original thesis not anticipated here — as long as it demonstrates argumentative rigor, correct use of sources, and autonomous critical reflection.*

1. Expected Thesis Position(s)

The central question admits multiple legitimate positions. Below are argumentative paths representing robust and distinct directions.

> ***Author note:*** *The recommended range is 2 to 4 theses. If the assignment has only one defensible answer, keep just Thesis A and adapt the wording. A third thesis often works best as a synthetic or dialectical position.*

1.1. Thesis A — {Short label}

{Full thesis statement, in one or two sentences. It should take a clear position on the central question.}

Main argumentative line: {One paragraph describing how this thesis sustains itself: the central claim, supporting reasoning, implicit framework, and what makes it defensible.}

Concepts mobilized: {Course concept 1} ({brief connection}); {Course concept 2} ({brief connection}); {Course concept 3} ({brief connection}).

1.2. Thesis B — {Short label}

{Full thesis statement — a meaningfully different position from Thesis A, not just a softer variant.}

Main argumentative line: {One paragraph describing the central claim, supporting reasoning, implicit framework, and what makes it defensible.}

Concepts mobilized: {Course concept 1} ({brief connection}); {Course concept 2} ({brief connection}); {Course concept 3} ({brief connection}).

1.3. Thesis C — {Synthetic/dialectical position, optional}

{Full thesis statement — a meaningfully different position from Thesis A, not just a softer variant.}

Main argumentative line: {Show explicitly how this position produces a new insight that neither A nor B alone can reach.}

Concepts mobilized: {Course concept 1} ({brief connection}); {Course concept 2} ({brief connection}); {Course concept 3} ({brief connection}).

2. Key Concepts and Their Correct Use

> ***Author note:*** *Include 3 to 5 key concepts per project. For each concept: precise definition + example of correct application + common pitfall. The pitfall section is critical — it tells the evaluator what to watch for.*

2.1. A — {Concept Name}

How to define/explain: {One paragraph defining the concept precisely. Include its theoretical origin or author when relevant. Define its scope and limits.}

Correct use (example): {A verbatim example paragraph showing the concept doing actual argumentative work — not just being named, but actively grounding a claim.}

Common pitfall: {The most frequent way students misuse this concept: over-application, decorative citation, missing nuance, ignoring critiques, or conflating related concepts. Be concrete.}

2.2. B — {Concept Name}

How to define/explain: {One paragraph defining the concept precisely. Include its theoretical origin or author when relevant. Define its scope and limits.}

Correct use (example): {A verbatim example paragraph showing the concept doing actual argumentative work and grounding a claim.}

Concepts mobilized: {Course concept 1} ({brief connection}); {Course concept 2} ({brief connection}); {Course concept 3} ({brief connection}).

2.3. C — {Concept Name}

How to define/explain: {One paragraph defining the concept precisely. Include its theoretical origin or author when relevant. Define its scope and limits.}

Correct use (example): {A verbatim example paragraph showing the concept doing actual argumentative work and grounding a claim.}

Concepts mobilized: {Course concept 1} ({brief connection}); {Course concept 2} ({brief connection}); {Course concept 3} ({brief connection}).

3. Model Argumentative Structure

The outline below illustrates the structure of an excellent-level essay, regardless of which thesis the student chooses.

> ***Author note:*** *Word ranges are suggestions calibrated for a {total length}-word essay. Adjust proportionally if the assignment is shorter or longer.*

3.1. Introduction ({X}–{Y} words)

Contemporary hook: {Describe what kind of opening works for this project — a concrete example, striking data point, scene from culture, or current event. Adapt to the project's domain.}

Bridge to subject: {How the hook should pivot into the project's central question.}

Explicit thesis: The author's position in one or two clear, specific sentences, positioned directly against the central question.

3.2. Development ({X}–{Y} words)

Suggested: 2–3 paragraphs, each with a central argument.

Paragraph 1: {Function and how it advances the thesis.}

Paragraph 2: {Function and how it advances the thesis.}

Paragraph 3: {Function and how it advances the thesis.}

3.3. Counterargument ({X}–{Y} words)

An excellent essay does not merely mention an objection: it presents the strongest counterargument with intellectual honesty and responds consistently.

> ***Author note:*** *List the best counterargument for each thesis option, with a suggested response. The student should not feel that confronting the objection weakens their position — done well, it strengthens it.*

If thesis is A: {Strongest objection and possible response.}

If thesis is B: {Strongest objection and possible response.}

If thesis is C: {Strongest objection and possible response.}

3.4. Conclusion ({X}–{Y} words)

Restatement of thesis: Reaffirm the position in light of the full argumentation, without repeating the introduction verbatim.

Implications: {What this reflection reveals about something larger — the field, how we think, or a contemporary issue.}

Opening (no new information): A provocative question or forward-looking gesture that broadens the horizon without introducing arguments not developed in the essay.

3.5. Evidence and Data Types to Mobilize

- Specific passages from primary sources.
- Theoretical concepts with citation to the original work.
- Documented contemporary examples: news, official reports, data, or cultural productions.
- References to academic debates covered in the course.

4. Rubric — Excellent Level

Detailed description of what the evaluator should observe in an essay rated “Excellent” on each criterion.

> ***Author note:*** *The criteria below cover the standard dimensions of argumentative essays. Remove or adapt criteria that do not apply to the specific project.*

4.1. Thesis clarity

What is observed at the EXCELLENT level: The thesis appears in the introduction unequivocally. It is specific and directly engages the project's central tension. The entire essay is organized around it.

4.2. Use of course concepts

What is observed at the EXCELLENT level: Required references are defined precisely before application. Concepts are integrated organically into the argument, and their limits are recognized.

4.3. Quality of argumentation

What is observed at the EXCELLENT level: Each paragraph opens with a clear claim, sustains it with evidence, and connects back to the thesis. Logical progression is clear and the student analyzes rather than merely describes.

4.4. Counterarguments

What is observed at the EXCELLENT level: The student presents the strongest objection to their position and responds with a substantial argument. The thesis emerges stronger from confronting the critique.

4.5. Originality of analysis

What is observed at the EXCELLENT level: The essay goes beyond reproducing course materials. The student offers original connections, asks new questions, or proposes a novel interpretation from familiar evidence.

4.6. Connection to the contemporary

What is observed at the EXCELLENT level: The dialogue between subject and present is specific and grounded. Concrete contemporary examples illuminate both sides.

4.7. Structure and academic norms

What is observed at the EXCELLENT level: Cohesive structure with fluid transitions. Citations follow the established academic pattern, and references are complete and correctly formatted.

5. Common Argumentative Errors

The errors below are the most frequent in this type of project. The evaluator should watch for them.

> ***Author note:*** *Errors 2, 3, and 4 apply to almost any argumentative project. Errors 1 and 5 are project-specific slots where particular risks may be described.*

5.1. {Project-specific error}

How it appears in the text: {Verbatim-style example showing the error in action.}

How to correct: {Concrete, actionable advice on what to do differently.}

5.2. Strawman fallacy

How it appears in the text: The student presents the counterargument in its weakest, easiest-to-refute form instead of confronting the strongest version.

How to correct: Reformulate the counterargument as an intelligent interlocutor would present it, then respond to that reason.

5.3. Decorative use of concepts

How it appears in the text: The student names a concept but never applies it — it appears as a loose citation, disconnected from the argument.

How to correct: After citing a concept, explain exactly how it grounds the specific argument of that paragraph.

5.4. False equivalence

How it appears in the text: The student treats as equivalent two phenomena that operate in radically different contexts or functions.

How to correct: Make differences explicit before drawing parallels. Use precise comparative language. Comparison is not equation.

5.5. {Project-specific error}

How it appears in the text: {Example}

How to correct: {Correction}

> ***Final reminder:*** *The quality of reasoning is always more important than the position defended. An essay that defends an unpopular thesis with rigor outperforms one that defends a consensus thesis superficially.*